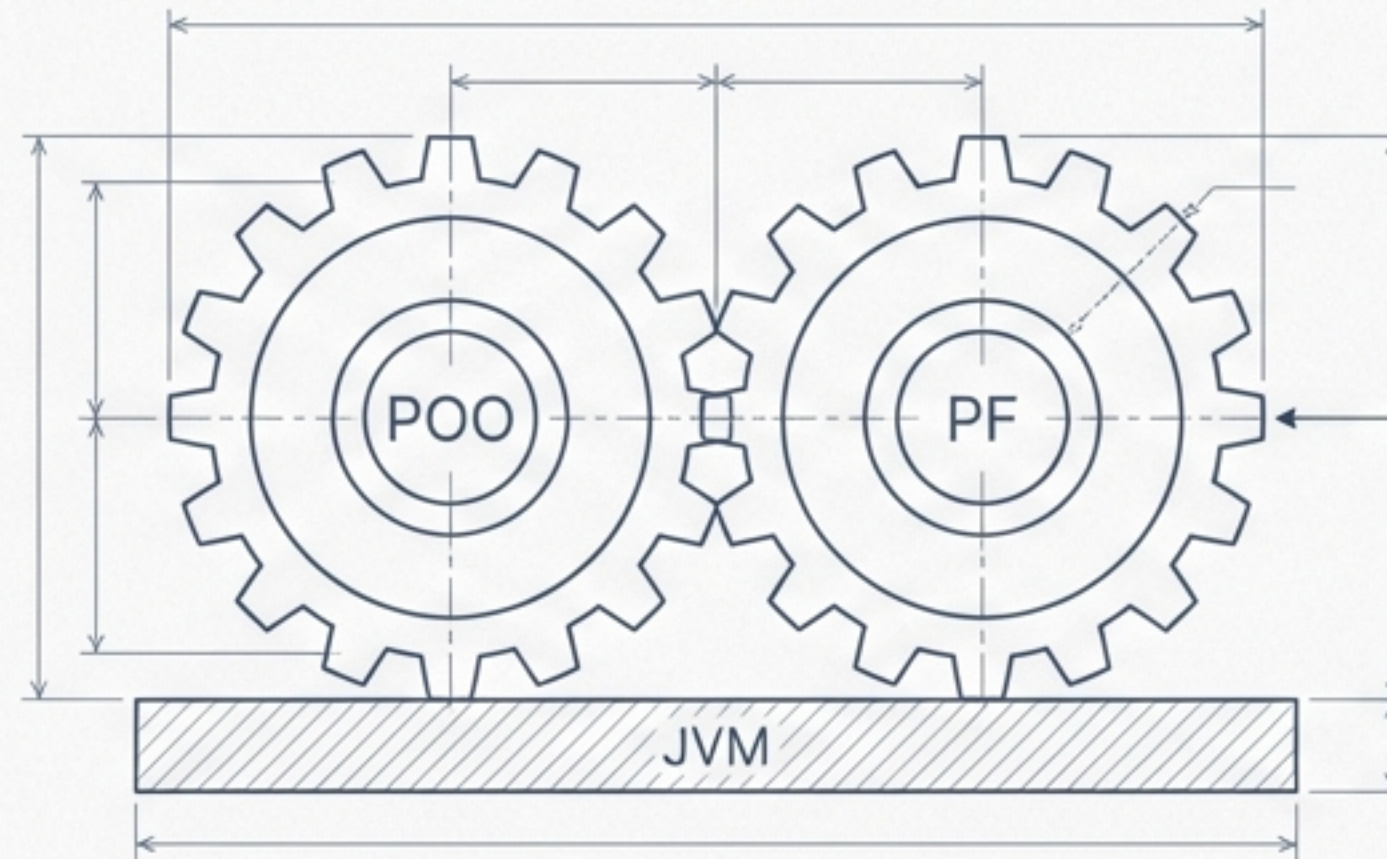


Dominando Scala 3 de Forma Pragmática

De la Sintaxis Segura a la Arquitectura Industrial



Scala es un lenguaje multiparadigma. El desarrollador puede elegir el paradigma más apropiado, como modelar datos con objetos y procesarlos con funciones puras para una mejor paralelización.

- Esta presentación interactiva demuestra cómo la filosofía de Scala 3 resuelve problemas complejos de la ingeniería de software moderna.
- Utilizaremos este *notebook* para ejecutar código en vivo, mostrando cómo las abstracciones de Scala eliminan la verbosidad de Java y la inseguridad en *runtime* de Python.
- El objetivo: construir software seguro, conciso y de alto rendimiento, verificable por el compilador.

El Cimiento: Inmutabilidad por Defecto y Concisión por Diseño

Menos Estado Mutable, Menos Problemas

En Scala, `val` (valor inmutable) es la norma, no `var` (variable mutable). La inmutabilidad es el secreto para la concurrencia segura: si los datos no cambian, no hay conflictos de acceso.

```
val x = 10 // Preferido
var y = 20 // Usar con precaución
```



`case class`: El Modelo de Datos Perfecto

Las `case class` son el estándar para modelar datos inmutables. Eliminan el código repetitivo (*boilerplate*) al generar automáticamente métodos esenciales como `equals`, `hashCode`, `toString` y, crucialmente, `copy`.

Java (Antes)

```
// public class Person {
//     private final String name;
//     private final int age;
//     public Person(String name, int age) { this.name = name; this.age = age; }
//     public String getName() { return name; }
//     public int getAge() { return age; }
//     @Override public boolean equals(Object o) { ... }
//     @Override public int hashCode() { ... }
//     @Override public String toString() { ... }
//     // ... y así sucesivamente
// }
```



Compilador genera el resto

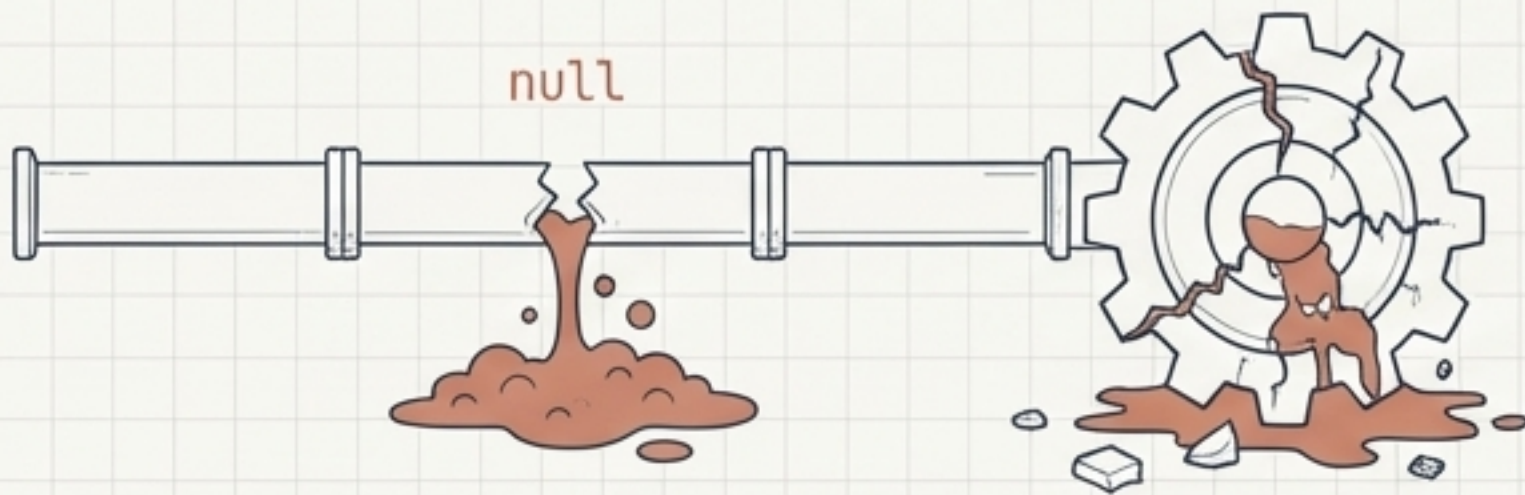
Scala (Ahora)

```
case class Persona(nombre: String, edad: Int)
```


Adiós al `null`: Tratar la Ausencia como un Valor, no como una Crisis

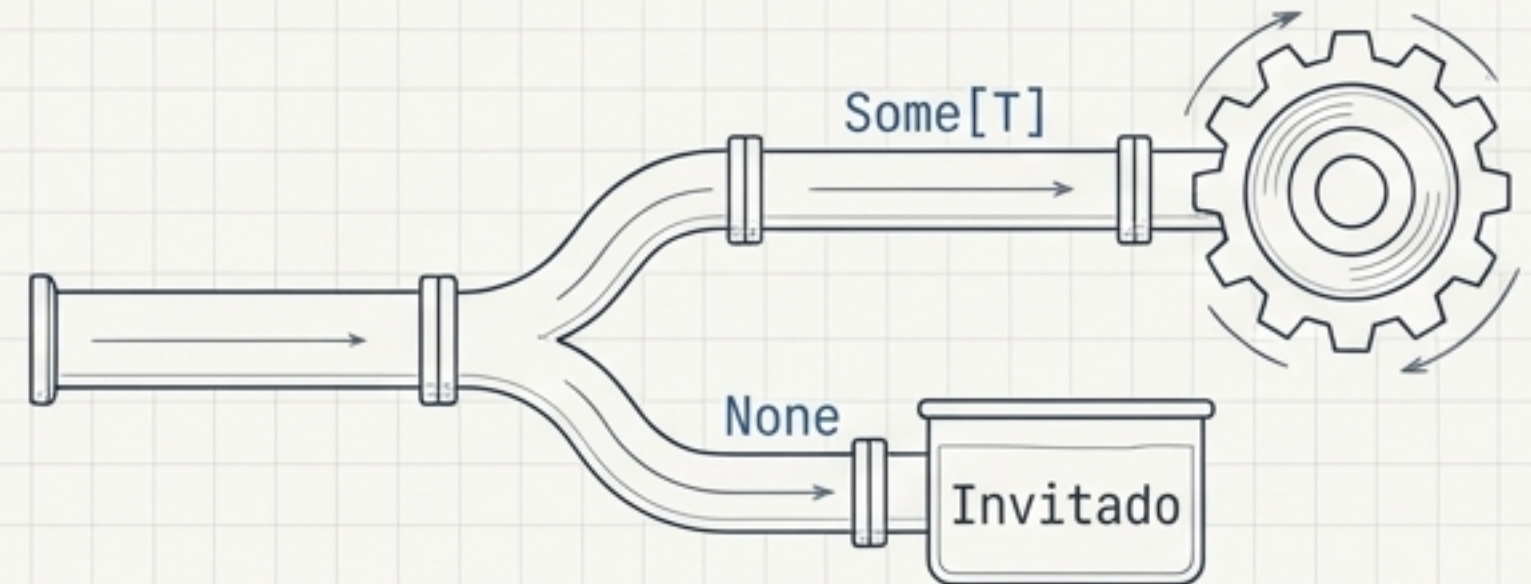
- El `NullPointerException` es una fuente constante de fallos en producción. **Scala** lo ataca de raíz con `Option[T]`.
- `Option[T]` es un contenedor que modela explícitamente la **presencia** de un valor (`Some[T]`) o su **ausencia** (`None`).
- El compilador te *obliga* a manejar la posibilidad de ausencia, convirtiendo un **error de runtime** en una **garantía en tiempo de compilación**.

Python / Java: Un Riesgo Latente



```
# def obtener_nombre(usuario):  
#     return usuario.nombre  
#  
# obtener_nombre(None) # AttributeError!
```

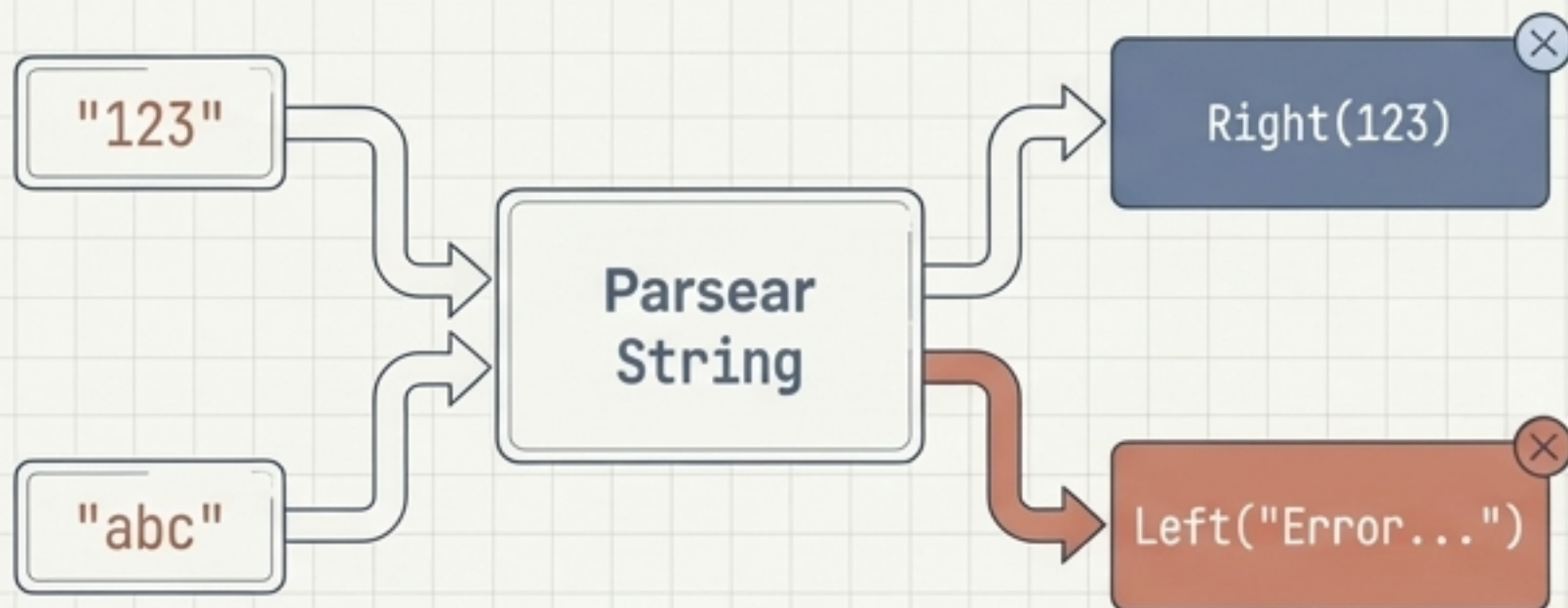
Scala: Seguridad por Diseño



```
// El flujo de ejecución es seguro y explícito.  
val saludo = usuarioAusente  
  .map(_.nombre)  
  .getOrElse("Invitado") // Flujo controlado, sin excepciones.
```


Los Errores son Información, no Excepciones

- Las excepciones rompen el flujo de control y son difíciles de razonar. **Scala promueve el manejo de fallos como valores tipados usando `Either[Error, Exito]`.**
- `Either` tiene dos posibles formas: `Left[Error]` para el caso de fallo (el error es un dato) y `Right[Exito]` para el caso de éxito.
- Esto permite **encadenar operaciones que pueden fallar**, donde la cadena se **detiene y propaga el primer error sin lanzar una excepción**.



Definición de la Función

```
def parsearNumero(s: String): Either[String, Int] = {  
  try {  
    Right(s.toInt)  
  } catch {  
    case e: NumberFormatException =>  
      Left(s"Error: '$s' no es un número válido.")  
  }  
}
```

Demostración de los resultados

```
val exito = parsearNumero("123")  
// resultado: Right(123)  
val fallo = parsearNumero("abc")  
// resultado: Left(Error: 'abc' no es un número válido.)
```

Encadenamiento Seguro

```
println(exito.map(_ * 2)) // Imprime: Right(246)  
println(fallo.map(_ * 2)) // Imprime: Left(Error: 'abc' no  
es un número válido.)
```

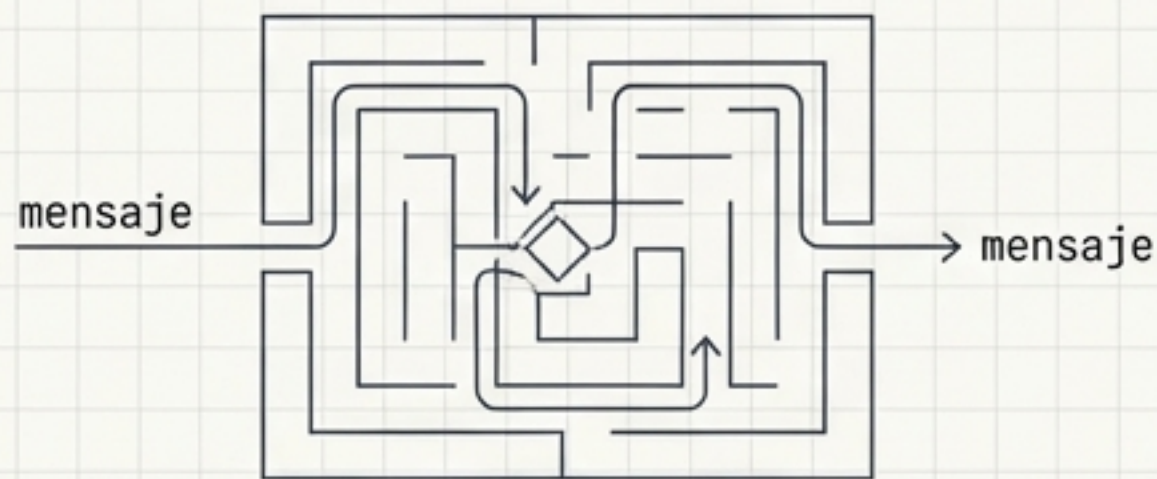
Observa cómo `.map` solo opera sobre el caso `Right`. El error se propaga de forma segura sin detener el programa.

El Código que Fluye: Todo es una Expresión

A diferencia de Java, donde `if-else` es una sentencia, en Scala es una **expresión** que evalúa a un valor. Lo mismo aplica para bloques de código `{}` y `match`.

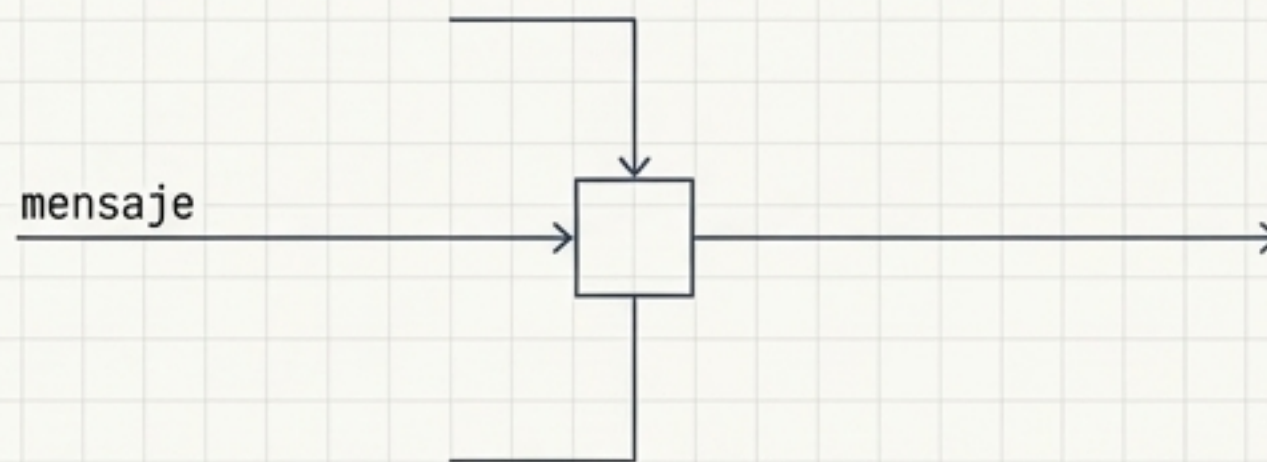
- ✓ Elimina la necesidad de variables temporales mutables (`var`).
- ✓ Reduce o elimina el uso de `return`, haciendo el código más fácil de seguir.
- ✓ El operador ternario de Java (`cond ? a : b`) es innecesario.

Estilo Imperativo (Java/Python)



```
// String mensaje;  
// if (usuarioExiste) {  
//     mensaje = "Bienvenido";  
// } else {  
//     mensaje = "Regístrese";  
// }
```

Estilo Expresivo (Scala)



```
val usuarioExiste = true  
val mensaje = if (usuarioExiste) "Bienvenido" else "Regístrese"  
println(mensaje)
```


Control de Flujo Inteligente: *Pattern Matching*

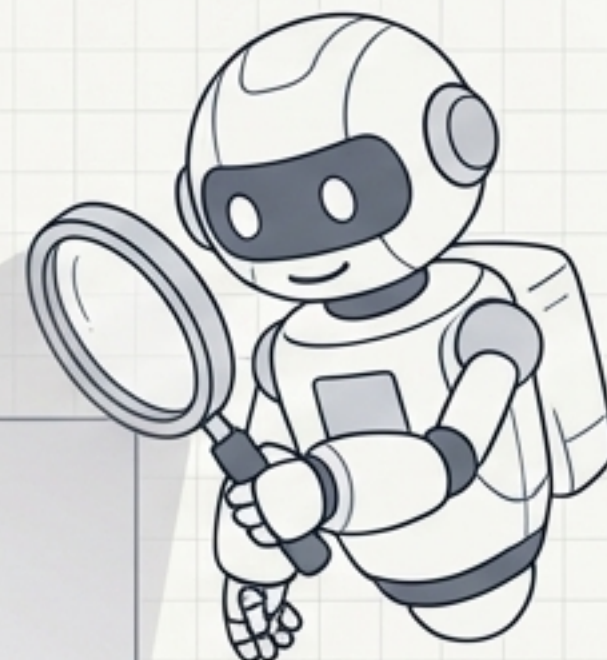
El *pattern matching* es la navaja suiza de Scala para la toma de decisiones. Supera al `switch` tradicional al permitir:

1. **Desestructurar datos:** Extraer valores de `case class` (`case Persona(n, e)`).
2. **Añadir condiciones:** Usar guardas `if` dentro de un caso.
3. **Garantizar totalidad (Comprobación de Exhaustividad):** El compilador te advierte si no has cubierto todos los casos posibles.

```
enum Comando {  
  case Iniciar, Detener, Pausar  
}
```

```
import Comando._  
  
def ejecutar(cmd: Comando): String =  
  cmd match {  
    case Iniciar => "Iniciando sistema..."  
    case Detener => "Deteniendo sistema..."  
  }  
}
```

.....● **match may not be exhaustive.**
It would fail on the following input: Pausar

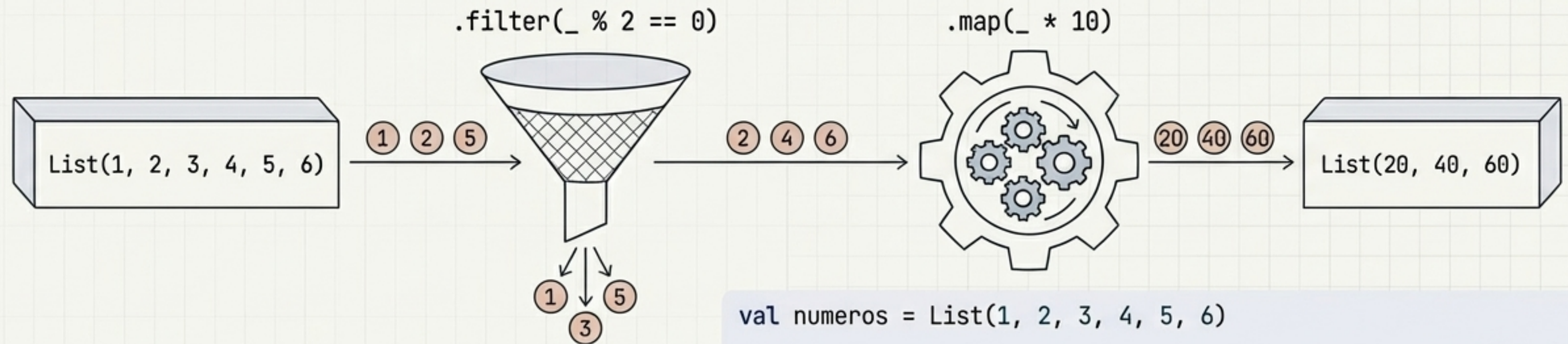


El compilador es tu mejor ingeniero de QA. Previene errores lógicos antes de que lleguen a producción.

De Bucles Imperativos a Tuberías (*Pipelines*) de Datos

En lugar de bucles ``for`` o ``while`` para iterar, Scala utiliza **Funciones de Orden Superior** (HOFs) como ``map``, ``filter`` y ``flatMap`` para transformar colecciones.

El código se lee como un **pipeline de transformaciones de datos**: 'toma esta lista, filtra los pares, luego multiplica cada uno por diez'. Este enfoque es **declarativo** ('qué' hacer, no 'cómo'), más conciso y, al operar sobre colecciones inmutables, inherentemente más seguro para la paralelización.



```
val numeros = List(1, 2, 3, 4, 5, 6)

val resultado = numeros
  .filter(_ % 2 == 0) // Quédate con los pares: List(2, 4, 6)
  .map(_ * 10)        // Multiplícalos por 10: List(20, 40, 60)

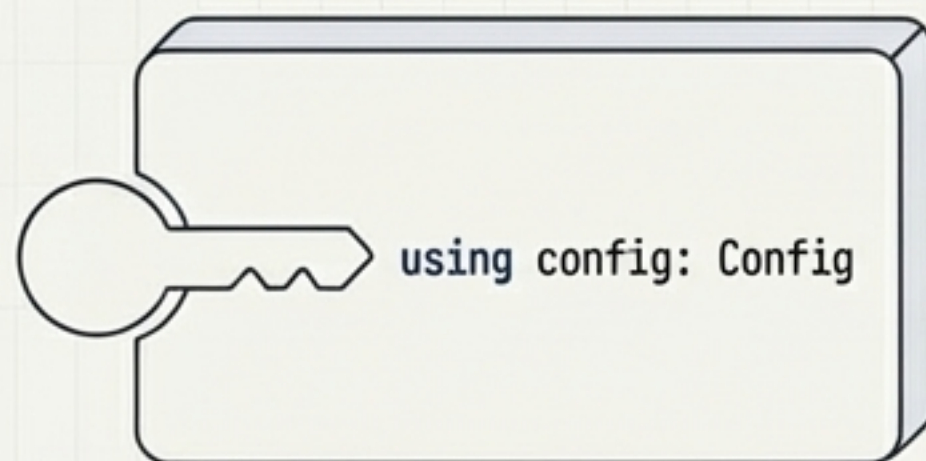
println(resultado)
```


El Compilador como tu Gestor de Dependencias: *`given/using`*

Las Abstracciones Contextuales (*given/using*) son la evolución de los *`implicit`* de Scala 2. Permiten inyectar dependencias o 'capacidades' de forma automática y segura.

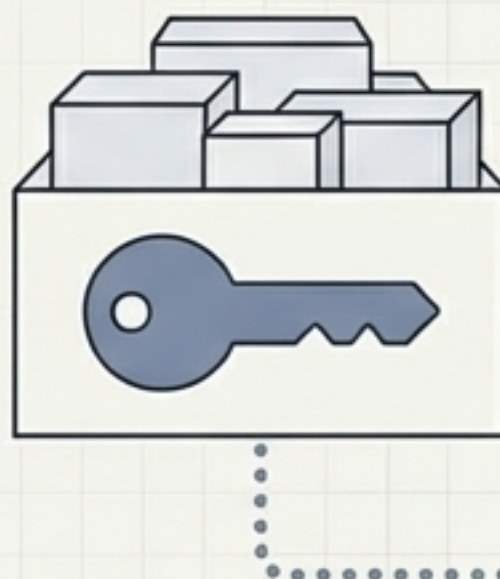
- **given:** "Proporciona un valor canónico o una instancia de un *`trait`* en un contexto determinado."
- **using:** "Declara que una función requiere una dependencia de cierto tipo. El compilador la buscará y la inyectará automáticamente."

PASO 1: REQUERIR (*using*)



```
trait Config { def url: String }  
def conectar(using config: Config): Unit = { ... }
```

PASO 2: PROVEER (*given*)



```
given configDePrueba: Config with {  
  val url = "..."  
}
```

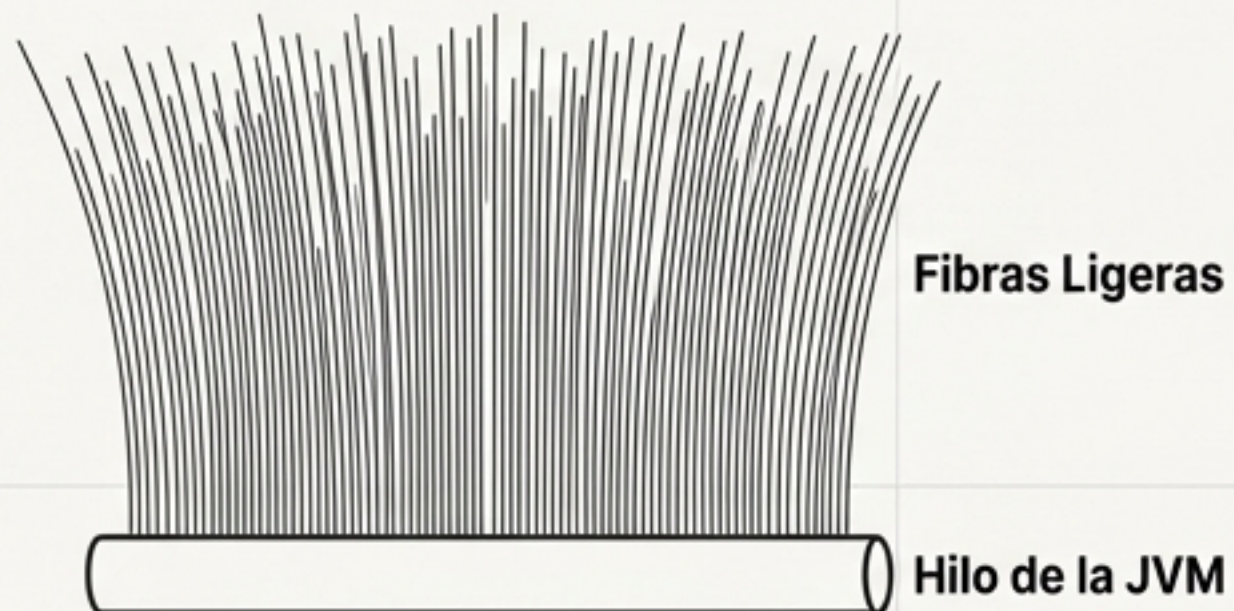
PASO 3: INYECTAR (compilador)



```
// llamamos a conectar() sin pasar la configuración.  
// El compilador lo hace por nosotros.  
conectar()
```


Construyendo a Escala: Concurrency Masiva y Resiliencia

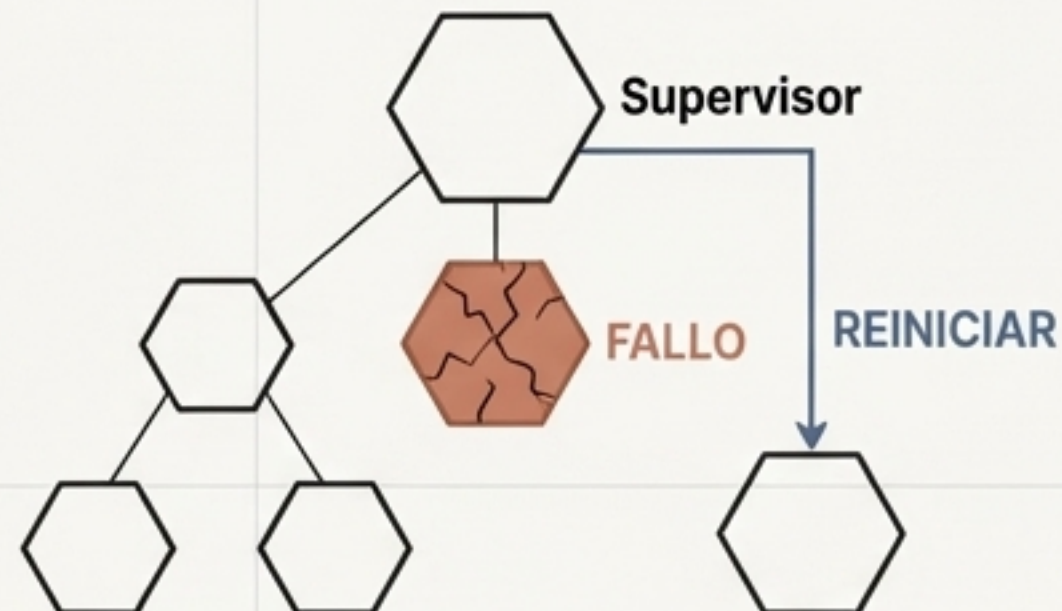
Fibras - Concurrency Eficiente (ZIO / Cats Effect)



Las Fibras son "hilos virtuales" extremadamente ligeros, gestionados por el runtime de Scala, no por el sistema operativo.

Permiten manejar cientos de miles o millones de operaciones concurrentes con un uso de recursos mínimo, superando ampliamente a los hilos pesados de la JVM.

Modelo de Actores - Resiliencia por Diseño (Pekko)



Un Actor es una unidad aislada de estado y comportamiento. Se comunican solo a través de mensajes inmutables.

El patrón clave es la Supervisión: en lugar de manejar errores internamente, un actor puede fallar ('Let-It-Crash') y dejar que su supervisor decida cómo recuperarse. Esto crea sistemas auto-reparables.

```
// Concepto de un Actor Contador en Pekko
// object Contador {
//   sealed trait Comando
//   case object Incrementar extends Comando
//
//   // El comportamiento define cómo reaccionar a los mensajes
//   def apply(): Behavior[Comando] = ???
// }
```


Casos de Éxito: Cuando la Falla No Es Una Opción

verizon[✓]

Doble Rendimiento con la Mitad de Hardware

Problema

Una plataforma monolítica heredada no podía soportar picos de tráfico (lanzamientos de iPhone).

Solución

Re-arquitectura con microservicios reactivos sobre Akka (predecesor de Pekko).

+750%

Capacidad de procesamiento de pedidos

/ 2

Costo Total de Propiedad (TCO)

+235%

Ventas en la nueva plataforma

Walmart  Canada

Conquistando el Black Friday

Problema

La infraestructura heredada fallaba bajo el tráfico extremo del Black Friday.

Solución

Reconstrucción de su plataforma con Scala y la Plataforma Akka.

+98%

Pedidos a través de móvil

-36%

Tiempos de carga durante picos

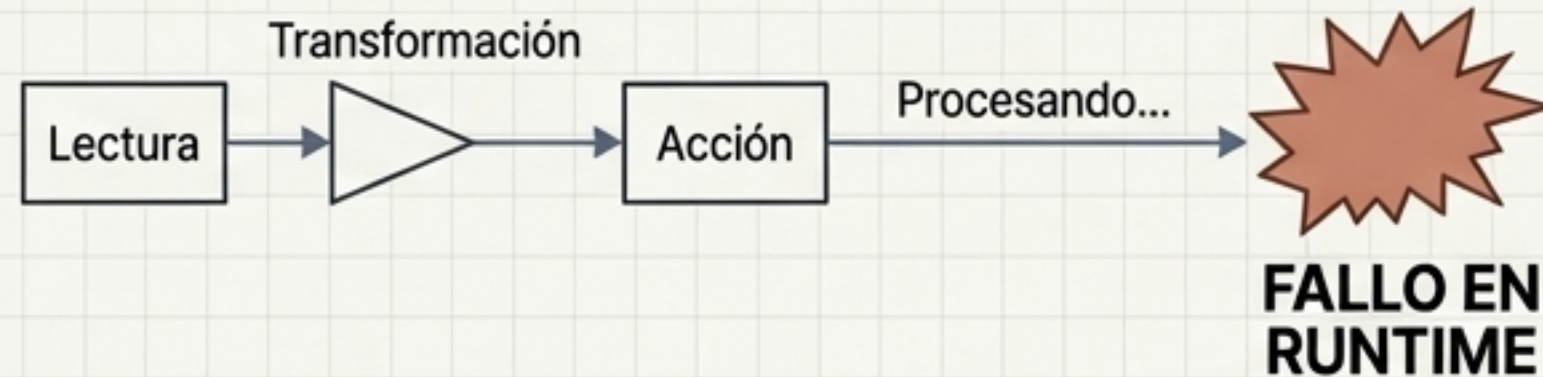
-50%

Costos de infraestructura web

Ingeniería de Datos con Esteroides: El Poder de Scala y Apache Spark

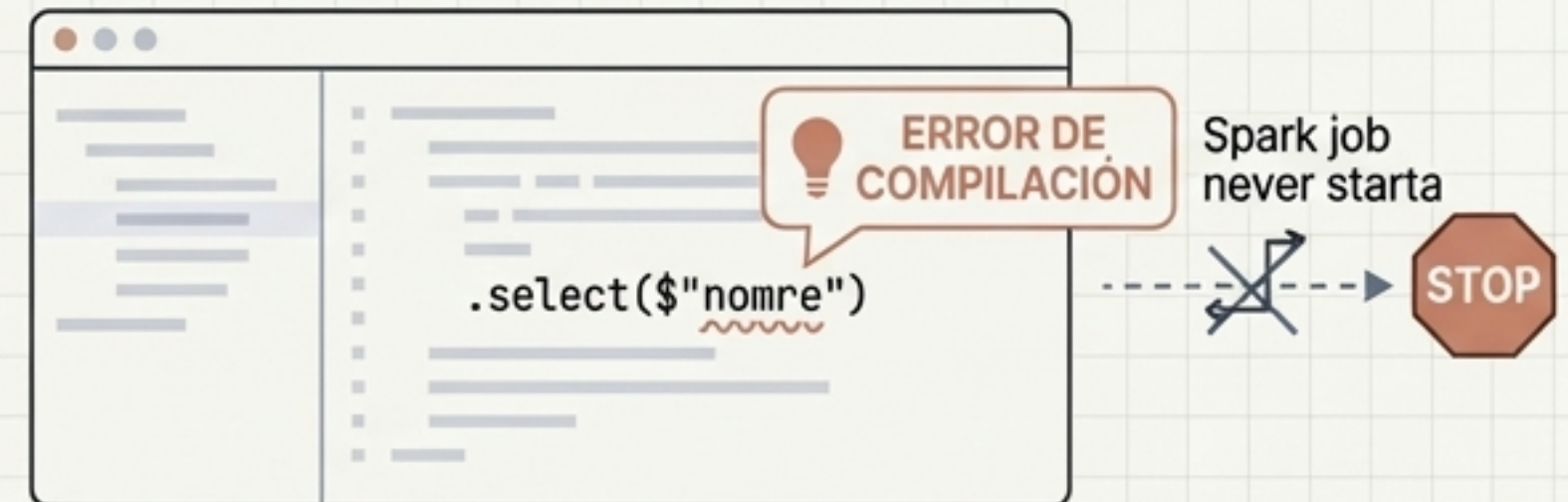
- Scala es el lenguaje de primera clase para Apache Spark. La ventaja clave es la API tipada: `Dataset[T]`.
- Gracias a las `case class`, los `Dataset` en Scala proporcionan seguridad de tipos para el esquema de datos.
- Un error en un nombre de columna, que en PySpark solo se descubriría en *runtime*, en Scala es un **error de compilación** detectado al instante.

PySpark - El Riesgo del Runtime



```
# # Error tipográfico en el nombre de la columna ('nomre')
# df.select("nomre").show()
#
# # AnalysisException: Column 'nomre' does not exist.
# # Este error SOLO se detecta al final del job.
```

Scala Spark - La Seguridad de la Compilación

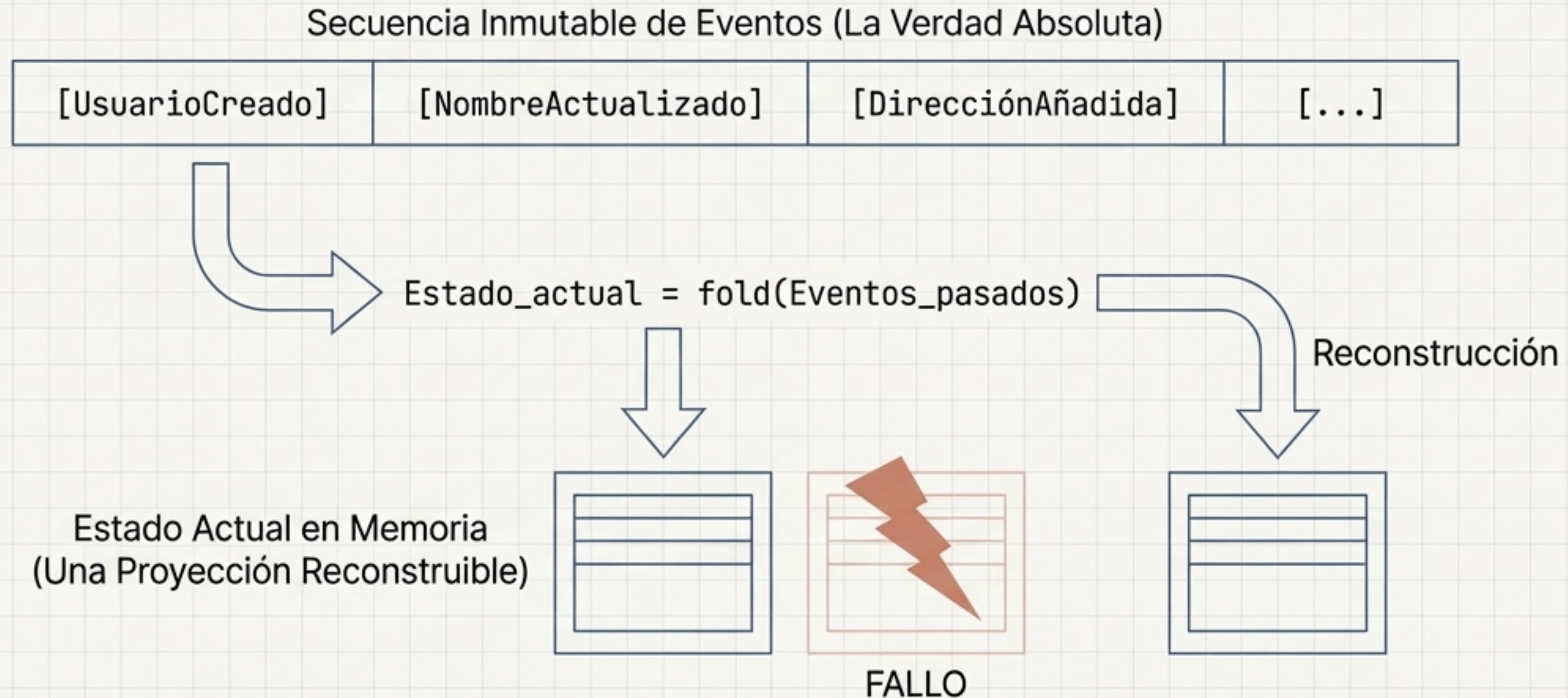


```
// case class Persona(nombre: String, edad: Int)
// val ds: Dataset[Persona] = ...
//
// ds.select($"nomre") // <-- ERROR DE COMPILACIÓN
// error: value nomre is not a member of String
```

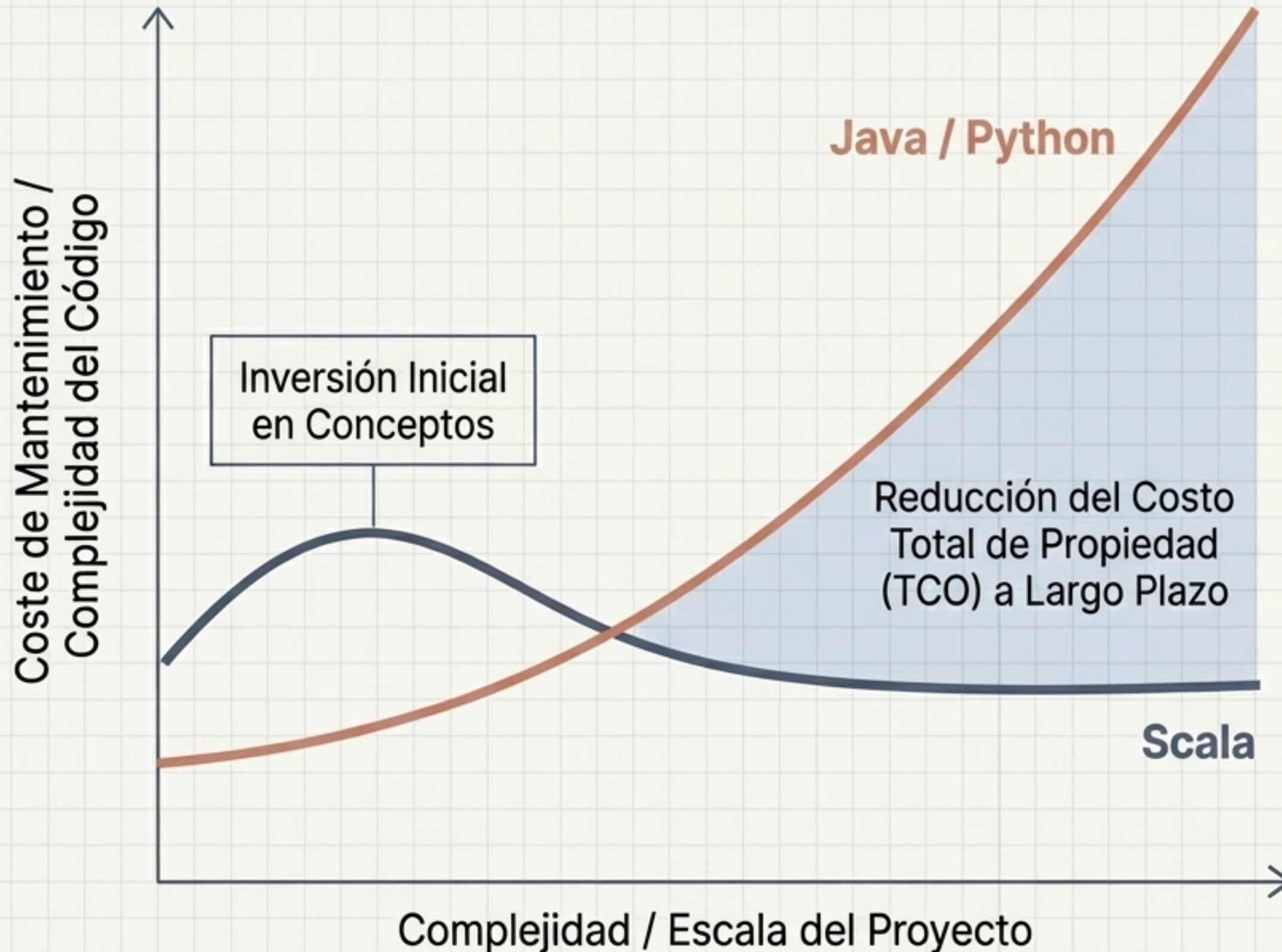

Sistemas Inmortales: Reconstruyendo el Estado con *Event Sourcing*

El patrón **Event Sourcing**, implementado en Pekko Persistence, es la clave para la resiliencia en sistemas distribuidos. En lugar de almacenar el estado actual, se persiste una secuencia inmutable de **eventos** que lo han modificado. El estado es una consecuencia de la historia.

Ventaja clave: En caso de fallo, el estado de un actor puede reconstruirse perfectamente desde cero simplemente reproduciendo sus eventos.



La Curva de Aprendizaje es una Inversión, no un Coste



La 'simplicidad' que ofrece Scala no está en la facilidad de aprendizaje inicial, sino **en la estabilidad operativa, la corrección y la resiliencia** del sistema final.

Representa una inversión inicial para una reducción a largo plazo de la complejidad operativa (menos bugs críticos, mayor tiempo de actividad, menos "apagar incendios").

Scala 3: Tu Socio para Construir Software Seguro, Conciso y Escalable



SEGURO POR DISEÑO

- El compilador es tu aliado.
- Previene errores de nulos, casos olvidados en la lógica y tipos de datos incorrectos antes de que el código llegue a producción.



CONCISO Y EXPRESIVO

- El código se centra en la lógica de negocio.
- Las ``case class`` y las expresiones funcionales eliminan el código repetitivo y superfluo.



ESCALABLE POR NATURALEZA

- Construido para la era multi-core y distribuida.
- La inmutabilidad y las abstracciones de concurrencia como Fibras y Actores son la base para sistemas de alto rendimiento.

Has visto cómo cada característica de Scala 3 es un bloque de construcción. Juntos, te permiten diseñar sistemas que antes eran prohibitivamente complejos, con un nivel de confianza y seguridad que otros ecosistemas no pueden igualar.

¿Listo para Construir?

Aprendizaje Guiado

Subtítulo: Cursos en Video

Rock the JVM (Cursos prácticos y avanzados)

DevInsideYou (Cientos de horas de contenido gratuito en YouTube)

Subtítulo: Ejercicios Interactivos

Scala Exercises (Ejercicios prácticos de 47 Degrees)

Exercism (Plataforma de ejercicios con mentoría)

Referencia Técnica

Subtítulo: Documentación Oficial

Scala 3 Book

Tour of Scala

Tu Siguiente Paso: Un Desafío Práctico

Toma una función imperativa con bucles de un proyecto actual (en Java, Python, etc.) y refactorízala a un estilo funcional en Scala usando ``map`` y ``filter``.

Observa la diferencia en claridad, concisión y, sobre todo, en la seguridad que proporciona el no tener estado mutable.